

PRD - AI Requirements Quality Evaluator

1. Executive Summary

Software delivery always comes back to one simple truth: teams can only build as clearly as we ask them to. Across my work in Program Management, Product, and Engineering Management, I've seen how fast a project can drift when requirements are vague or overloaded with assumptions. The research backs up what most engineers already know - ambiguity slows teams down, introduces defects, and forces people to interpret instead of deliver. (See: <https://arxiv.org/abs/2404.11106>)

This project introduces an extensible **AI Requirements Quality Evaluator** built to bring clarity upstream. It doesn't try to "rewrite" requirements for teams. Instead, it evaluates them with structure and consistency so PMs, engineers, and cross-functional partners can start from the same baseline. The tool is intentionally lightweight, cost-efficient, and designed to plug into the tools teams already use (Jira, GitHub, CI/CD). Daily Bedrock call limits help keep the system predictable, safe, and portfolio-friendly.

2. Problem Statement & Strategic Opportunity

Requirements remain one of the most fragile parts of software development:

- Ambiguous and low-quality requirements are a primary cause of defects and delivery delays. (<https://arxiv.org/abs/2404.11106>)
- Distributed and outsourced teams show significantly higher failure rates when requirements lack clarity. (<https://www.sciencedirect.com/science/article/pii/S0950584925001624>)
- Defects originating in the requirements phase are exponentially more expensive to fix later. (NASA: <https://ntrs.nasa.gov/api/citations/20100036670/downloads/20100036670.pdf>)
- Existing AI tools focus on generation rather than **deterministic, measurable evaluation**.

This creates a strong opening for a platform that enforces requirement hygiene, supports engineering rigor, and offers auditable outputs that leadership can trust.

3. Strategic Alignment & Market Timing

AI is showing up across engineering organizations, but most teams still struggle to separate hype from durable value. Leaders aren't asking for bigger models - they're asking for models they can trust. That requires guardrails, clear evaluation criteria, and predictable behavior.

This evaluator supports those goals directly. It reduces ambiguity before work begins, gives Engineering Managers cleaner inputs for planning, and helps Product teams keep stories aligned with intent. It also supports organizational priorities around consistency, visibility, and engineering throughput - without demanding new workflows or heavy change management.

4. Success Metrics

These metrics reflect the expectations of a Principal PM partnering closely with Engineering Managers to ensure organization-wide impact.

Organizational & Delivery Metrics (Cross-Functional)

- **30% reduction in requirement clarification loops**
 - Tracked via automated comment-pattern detection in Jira/Linear.
 - Joint PM/EM review with weekly dashboards and RCA triggers.
- **20–25% reduction in defects linked to poor requirements**
 - Validated through QA/Eg defect root-cause tagging.
 - Alerts generated when requirement-related defects exceed thresholds.
- **10–15% improvement in cycle time**
 - Measured through story lead-time/cycle-time SLIs.
 - Insights incorporated into sprint planning and capacity modeling.

AI Reliability & Model Evaluation Metrics

- **$\geq 85\%$ accuracy in ambiguity detection**
 - Weekly eval-suite execution against curated dataset.
 - Drift alerts triggered if accuracy drops $>5\%$ week over week.
- **$\geq 80\%$ accuracy in completeness/testability detection**
 - Automated diffs between expected vs. actual outputs.
- **$\leq 10\%$ false-positive rate**
 - FP/FN rates tracked across all evaluations.
- **$< 5\%$ hallucination rate**
 - JSON schema validation flags hallucinations.

- Spikes generate engineering follow-ups.

Platform Performance, Cost & Operational Metrics

- **P95 latency < 1 second**
 - Monitored through CloudWatch; regressions trigger EM/DevOps review.
 - **Cost < \ \$5/month**
 - AWS budget alerts at 70% and 90% thresholds.
 - **Daily Bedrock call limit enforced**
 - DynamoDB counters + CloudWatch alarms.
-

5. Target Users & Stakeholders

Primary Users

- Product Managers
- Engineering Managers
- Tech Leads
- QA/Test Engineers

Stakeholder Alignment

- **CTO:** Reliability, engineering throughput.
 - **Director of Engineering:** Cycle-time reduction, defect prevention.
 - **PMO/Head of Product:** Standardization across teams.
 - **Compliance/Security:** Safe and auditable AI usage.
-

6. Functional Requirements (FR)

1. Accept requirement/user-story text inputs.
 2. Produce structured evaluation (ambiguity, testability, completeness, assumptions).
 3. Output human-readable summary + JSON format.
 4. Evaluate AI results against curated dataset.
 5. Enforce user/IP-based daily API quota.
 6. Provide deterministic prompt templates.
 7. Return clear error messaging.
-

7. Non-Functional Requirements (NFR)

- Reliability: 99% uptime.
 - Performance: P95 under 1 second.
 - Cost: Operate within ~\$5/month.
 - Security: No persistent data.
 - Scalability: Entirely serverless.
 - Auditability: Detailed structured logs.
 - Abuse prevention: IP/user rate limiting.
-

8. Evaluation Framework

Eval Dataset

~30–40 annotated requirements capturing ambiguity, completeness, testability, assumptions, and metric usage.

Eval Logic

Evaluation compares expected vs. actual labels:

```
{
  "expected": { "ambiguity_detected": true },
  "actual": { "ambiguity_detected": false },
  "result": "false_negative"
}
```

Metrics include:

- Accuracy
- False positives and false negatives
- Week-over-week stability trends

Why It Matters

This shifts AI from an improvisational generator to a **measurable, reliable, and auditable platform component** - essential for enterprise adoption.

9. Architecture Overview

Components

- Frontend: S3 + CloudFront

- Backend: Lambda (Python)
- AI: Amazon Bedrock (Claude 3 Haiku)
- Rate Limiting: DynamoDB/CloudWatch
- API Gateway
- Observability: CloudWatch

System Flow

1. User submits input.
2. Lambda sends structured request to Bedrock.
3. Lambda validates, evaluates, and formats results.
4. Usage counters increment.
5. Throttling activated when limits are reached.

10. Risks & Mitigations

Risk	Mitigation
Model hallucination	Eval dataset + JSON schema enforcement
Domain-specific nuance	Domain-specific smell profiles
Abuse/cost risk	Daily call limits
Overly strict output	Prompt tuning + user feedback loop
Adoption friction	Low-friction Jira/GitHub integrations

11. Roadmap

Phase 1 - Portfolio MVP

- Static UI
- Bedrock-powered evaluator
- Structured JSON output
- Eval dataset (30 examples)
- API quota enforcement

Phase 2 - Workflow Integrations

- Jira plugin
- Slack requirement-refinement bot
- CI/CD quality gate for story acceptance

Phase 3 - Enterprise Platform

- Trend dashboards
 - Domain-specific requirement profiles
 - Versioned evaluation models
 - Governance controls & compliance logging
-

12. Research & References

- **Characterizing Requirements Smells (2024)** - ambiguity, unverifiability, incompleteness. <https://arxiv.org/abs/2404.11106>
 - **Natural-Language Requirements Testability Measurement (2024)** - smell-based testability metrics. <https://arxiv.org/abs/2403.17479>
 - **Automated Smell Detection in NL Requirements (2023)** - applied NLP detection. <https://arxiv.org/abs/2305.07097>
 - **NASA Error Cost Escalation Study** - defect cost increases over time. <https://ntrs.nasa.gov/api/citations/20100036670/downloads/20100036670.pdf>
 - **Cost of Requirement Errors** - financial impact of requirement defects. <https://www.stickyminds.com/article/what-cost-requirement-error>
-

13. Conclusion

This evaluator is meant to do one thing well: make requirements clearer, earlier. It gives teams a shared lens for quality, it reduces the back-and-forth that slows planning, and it prevents avoidable defects from slipping downstream. More importantly, it reflects how I approach building products and leading teams - with clarity, intentionality, and a focus on systems that make people's work easier, not harder.

It's a practical foundation for a larger **AI-driven requirements quality platform**, and it demonstrates my experience operating at the intersection of Product, Engineering, and Program Management - aligning teams, tightening feedback loops, and driving measurable improvement in how software gets built.